GCS THEORY Q&A

BY - SHUBHAM WADEKAR

Copyright © Shubham Wadekar. All rights reserved.

This material is for personal use only. No part may be copied, shared, resold, or published without written permission. Unauthorized distribution is strictly prohibited and may result in action.

For permissions or licensing, contact: [shubham.p.wadekar@gmail.com]

Disclaimer: This content is for educational purposes. Accuracy is attempted but not guaranteed.

No job outcome is promised.

© Shubham Wa

1. What are the best practices for naming buckets and folder structures in a GCS-based data lake?

When I design a GCS data lake, I try to make the bucket names and folder structures very clear and consistent so that anyone can understand what's inside. Buckets are global, so I usually include project name, environment, region, and data domain in the name. For example: project-prod-us-central1-datalake. This makes it easy to know which project and region the bucket belongs to.

Inside the bucket, I organize the folders by zones like raw, curated, and processed. Within each zone, I go deeper with hierarchy like zone/dataset/year/month/day/. This helps with partitioning and lifecycle management. For example: raw/sales/2025/09/11/filename.json.

The key best practices I follow are:

- Keep names lowercase, no spaces, use hyphens or underscores.
- Make bucket names unique and descriptive.
- Use a consistent folder structure so downstream jobs can easily automate file discovery.
- Avoid putting too many files in a single folder — instead use date-based partitioning.

2. Why is it useful to separate raw, curated, and processed zones in GCS?

Separating zones makes the data lake much more manageable and reduces confusion. Raw zone is where I store the original data exactly as it lands from the source, without changes. This is like a backup or source of truth — if anything goes wrong in processing, I can always go back to raw data.

Curated zone is where I keep cleaned and standardized data. This is where schema alignment, deduplication, and quality checks happen. Analysts and downstream systems usually read from curated because it's reliable and ready for use.

Processed zone is for aggregated or transformed data, like ready-to-use datasets for reporting, dashboards, or machine learning.

By separating zones, I can:

- Apply different access controls (for example, only engineers can see raw, analysts see curated/processed).
- Implement lifecycle policies differently (raw can move to cheaper storage, curated stays on standard).
- Simplify governance and auditing.

This layered structure ensures both flexibility and reliability.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

3. How do you decide between regional and multi-regional buckets for ETL pipelines?

For ETL pipelines, the choice depends on where the compute jobs run and what the access pattern looks like. If my Dataproc, Dataflow, or BigQuery jobs are running in a specific region like us-central1, then I prefer a regional bucket in the same region. This reduces latency and also cuts down network egress costs, because data stays within the same region.

Multi-regional buckets are useful when the data needs to be accessed globally with high availability, like if multiple teams in different continents are reading the same files. They are more expensive but provide better durability and availability across regions.

So the rule of thumb I follow is: for ETL pipelines that run in one region → use regional bucket. For datasets that are business-critical and need global access → use multi-regional.

4. When would you choose Nearline, Coldline, or Archive storage classes in data engineering?

These classes are all about optimizing cost based on how often I access the data.

- Nearline: I use this when data is accessed maybe once a month or less, like monthly reports or logs that are rarely read. It's cheaper than standard storage but still good for medium-term storage.
- Coldline: I use this for data I hardly ever access, like yearly audit data or compliance-related data that I may need once or twice in a year. It's cheaper but has higher access cost.
- Archive: This is for long-term storage that I almost never need, but must retain for compliance or regulatory reasons, like financial data that needs to be kept for 7 years. It's the cheapest, but retrieval takes time and costs more.

So, the decision is based on access frequency: the less I access the data, the colder the storage class I choose.

5. How would you design GCS buckets for cross-team or cross-project data pipelines?

When different teams or projects need to share data, I usually design buckets with clear boundaries and apply IAM policies carefully. One approach is to create central data lake buckets owned by a platform team, and then give other teams controlled access (read or write) only to the zones or folders they need.

For example, Team A may have write access to raw/teamA/ but read-only access to curated/finance/. This way, teams cannot overwrite each other's data.

If cross-project sharing is needed, I use service accounts and grant them roles like Storage Object Viewer or Storage Object Creator at the right folder level. Sometimes I also use VPC-SC (Service Controls) for stricter data boundaries.

Another important thing is to separate environments — for example, buckets for dev, test, and prod. This avoids accidental overwrite or exposure.

In short, the design is about centralizing storage but decentralizing access in a controlled way, so teams can collaborate safely without interfering with each other.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

6. What are common methods to batch ingest data from on-prem into GCS?

For batch ingestion from on-prem into GCS, there are several methods depending on the size and frequency of the data. For small or medium datasets, I can use gsutil or the gcloud CLI to copy files directly. For regular batch transfers, I usually configure **Storage Transfer Service** — it automates moving files from on-prem to GCS on a schedule, handles retries, and can deal with incremental changes.

For very large datasets, Transfer Appliance is another option. Google provides a physical device that can be filled with data on-prem and then shipped to Google, where the data is uploaded into GCS. This is useful when moving petabytes of data where network transfer is too slow.

Another method is setting up Data Transfer via VPN or Interconnect for secure high-throughput connectivity, and then automating ingestion jobs through scripts or orchestration tools like Composer.

So in short, I pick the method based on scale: gsutil for ad-hoc, Storage Transfer Service for automated scheduled transfers, and Transfer Appliance for massive one-time migrations.

7. How can Pub/Sub be used with Dataflow to ingest streaming data into GCS?

The usual pattern is to stream messages into Pub/Sub and then build a Dataflow pipeline to subscribe to the topic. Dataflow can apply any transformation or parsing needed, and then it writes the data into GCS as files.

For example, if logs are coming in real-time from an application, each log entry is published to Pub/Sub. Dataflow reads those messages, batches them into time windows (say, 5 minutes), and writes the batch as files in GCS like logs/yyyy/mm/dd/hh/.

This is useful because Pub/Sub handles scalability and durability of messages, while Dataflow provides flexible windowing and transformation before writing to GCS.

8. How can event-driven ingestion be achieved when files are uploaded to GCS?

Event-driven ingestion is achieved by using **Cloud Functions** or **Cloud Run** with GCS event notifications. Whenever a file is uploaded to a bucket, GCS can trigger a Pub/Sub event. That Pub/Sub event then triggers a Cloud Function or Cloud Run service, which can process the file or trigger a downstream pipeline.

For example, when a CSV file lands in a raw/ bucket, a Cloud Function can get triggered to validate schema and move it to the curated/ folder. Or it can start a Dataflow or Dataproc job to process the file.

This removes the need for polling and makes the system near real-time. It's a common pattern for event-driven pipelines.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

9. What are some best practices for bulk loading large datasets into GCS?

For bulk loads, I follow a few best practices:

- Parallelize uploads: Tools like gsutil -m cp or Storage Transfer Service support multi-threaded uploads, which speeds up large transfers.
- Split large files: Instead of one giant file, break data into chunks (like 100MB to 1GB per file). This improves performance in downstream processing because GCP services can parallelize better.
- Use regional buckets close to source or compute: This reduces latency and egress costs.
- Compress files: Using formats like Parquet, Avro, or GZIP saves storage cost and speeds up transfers.
- Retry and resume support: Use tools that can handle failures and resume without starting over.
- Pre-create folder structure: Organize by date or partitions before loading, so downstream jobs don't have to shuffle later.

These practices ensure faster, reliable, and more cost-effective ingestion at scale.

10. How can you design incremental ingestion strategies into GCS?

Incremental ingestion means I only bring in new or changed data instead of reloading everything. There are a few ways to design this:

- File-based watermarking: If files arrive daily, I can use file names or timestamps (like YYYY-MM-DD) to only ingest new files into GCS.
- Database change capture (CDC): For structured sources, I can capture only new inserts/updates using tools like Datastream, Debezium, or on-prem database logs, then write those deltas into GCS.
- Storage Transfer Service incremental option: It supports copying only files modified after the last transfer.
- Metadata tracking: Keep a record (in BigQuery or a metadata store) of the last processed file or batch, and only fetch beyond that.

The idea is to minimize data movement and cost, while ensuring the pipeline is idempotent (re-running doesn't duplicate data).

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

11. How can you load CSV or JSON files from GCS into BigQuery?

There are a few ways to load CSV or JSON files from GCS into BigQuery. The simplest is to use the BigQuery console, where I select "Create table," choose GCS as the source, and select CSV or JSON as the format. Another way is using the bq load command in the CLI or automating through API calls.

When loading, I can either define the schema manually or let BigQuery auto-detect it. I also configure options like field delimiter, header row skip, or null value handling for CSV. For JSON, I can load either line-delimited JSON (each row is a JSON object) or nested JSON for semi-structured data.

For recurring loads, I usually schedule them with Cloud Composer, Dataflow, or even use BigQuery scheduled queries if the staging is already in GCS.

12. What is a federated query in BigQuery, and how does it work with GCS files?

A federated query in BigQuery means I can query external data directly, without loading it into BigQuery storage. For GCS, this means I can define an **external table** that points to files in a bucket (like CSV, JSON, Parquet, or Avro). Then I can run SQL queries on top of that table.

BigQuery reads the data from GCS at query time. This is useful when the dataset is large and I don't want to pay storage costs in BigQuery, or when the data is updated frequently and I just want to query it as-is.

However, federated queries are usually slower and more expensive per query compared to native BigQuery tables, because the data has to be read fresh from GCS each time. That's why in production, I often query externally for exploration, but then load into BigQuery tables for repeated analytics.

13. How do you load data into partitioned and clustered BigQuery tables from GCS?

When loading into partitioned tables, I define the partitioning scheme — usually ingestion-time partitioning or column-based partitioning (like date). For example, if files are organized by date in GCS (/2025/09/11/), I can load them into a table partitioned by date column. This ensures queries later only scan relevant partitions.

For clustering, I specify one or more columns that will be used to physically organize the data (like customer_id, region, or status). Clustering works best when queries often filter or group by those fields.

The actual load job doesn't change much compared to normal loads — I just configure partitioning and clustering when I create the table, and then load GCS files into it. If I automate the ingestion, I usually map folder structures in GCS to partitions in BigQuery.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

14. How does BigQuery handle schema changes when loading data from GCS?

BigQuery is quite strict with schemas, but it allows some flexibility. If the schema in the file has extra fields that don't exist in the table, the load will fail unless I enable schema evolution (like allowing new nullable columns). Adding new columns is supported as long as they are optional or nullable.

If a field type changes (for example, string to integer), BigQuery will usually throw an error. Also, removing columns is not supported directly.

For managing schema changes, I usually design ingestion pipelines with options like:

- Using schema_update_options=ALLOW_FIELD_ADDITION for new columns.
- Keeping raw data in GCS and reprocessing if schema evolves.
- Using Dataflow or custom scripts to normalize schema before loading.

So schema drift has to be managed carefully, otherwise loads will break.

15. How can you optimize GCS-to-BigQuery load jobs for cost and performance?

The main optimizations I follow are:

- Use compressed and columnar formats: Instead of CSV or JSON, I prefer Avro or Parquet. They are faster to load and support nested data.
- Batch files smartly: Instead of loading thousands of tiny files, I combine them into larger files (100MB–1GB). Too many small files slow down load jobs.
- Partition and cluster: This reduces query cost later by scanning less data.
- Schedule loads in bulk: Instead of triggering per file, I group files for batch loads. This cuts down on job overhead and costs.
- Use regional buckets close to BigQuery dataset: This avoids cross-region network charges and reduces latency.
- Monitor job performance: Track load job logs in Cloud Logging to catch bottlenecks early.

By following these, I reduce both the load job cost and the downstream query cost in BigQuery.

16. How can Dataflow read and write Avro, Parquet, JSON, and CSV files from GCS?

Dataflow supports all these formats through built-in IO connectors. For reading, I can use TextIO for CSV and JSON (if they are line-delimited), and AvroIO or ParquetIO for Avro and Parquet files. Each IO automatically understands the format and schema where applicable.

For writing, it's similar: TextIO writes plain text files, AvroIO writes Avro files, and ParquetIO writes Parquet. JSON can also be written line by line using TextIO, or with custom formatting if it is nested JSON.

In practice, if I want structured and efficient storage, I prefer Avro or Parquet because they preserve schema and support nested data types. CSV and JSON are more common for raw ingestion but less efficient for analytics.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

17. What is the difference between batch and streaming pipelines using GCS as a sink?

In batch pipelines, the data is processed in finite chunks and written once at the end of the job. For example, reading a day's worth of logs and writing them into a single set of files in GCS. The output is predictable and usually written as one file per shard.

In streaming pipelines, the data is continuous, so Dataflow writes files incrementally. To avoid too many tiny files, it uses windowing and triggers to group events before writing. For example, I can set a 5-minute window so Dataflow writes one file every 5 minutes instead of per record.

So the main difference is that batch writes produce a fixed output at once, while streaming writes produce rolling outputs over time, controlled by windows.

18. What is schema-on-read vs schema-on-write when working with GCS and Dataflow?

Schema-on-write means the schema is enforced when writing the data. For example, if I use AvroIO to write into GCS, I must define the schema, and the data is stored in that structured format. Any mismatch will fail at write time.

Schema-on-read means the data is stored in raw form, like CSV or JSON, and the schema is applied only when reading later. For example, I may write raw JSON logs into GCS and later use Dataflow or BigQuery to define the schema during processing.

With GCS, both approaches are possible. If I need strong governance and consistent data, I prefer schema-on-write formats like Avro or Parquet. If I want flexibility for exploratory or raw storage, I use schema-on-read formats like JSON or CSV.

19. How does windowing affect writing streaming results to GCS?

In streaming, data is unbounded, so without windowing, Dataflow would try to write an infinite stream to GCS, which is not practical. Windowing breaks the stream into finite chunks based on time or event characteristics.

For example, if I set a fixed window of 10 minutes, Dataflow collects all records in that window and writes them together as a file in GCS. This avoids having thousands of tiny files and gives meaningful file boundaries like 2025-09-11-10min.avro.

Triggers can also be used along with windowing to control when partial results are written. So windowing directly impacts how files are organized, their size, and how efficiently downstream jobs can process them.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

20. How can you optimize Dataflow sinks when writing files into GCS?

A few strategies help optimize performance and cost:

- Use larger window sizes to reduce the number of small files. Too many tiny files create overhead in storage and downstream queries.
- Choose efficient formats like Avro or Parquet for structured data. They compress better and preserve schema.
- Use sharding wisely. Dataflow lets me control the number of output shards. More shards increase parallelism, but too many create small files. A balance is needed based on data size.
- Compress text outputs (GZIP) to reduce storage and network costs.
- Place GCS buckets in the same region as the Dataflow job to avoid cross-region egress.
- Monitor pipeline logs to catch backpressure or retries, which can cause duplicate file writes.

By tuning these, I can make sure the sink is efficient, cost-effective, and easy to use for downstream systems.

21. Why is GCS typically used instead of HDFS as storage in Dataproc?

The main reason is that GCS is fully managed and highly scalable, while HDFS is tied to the cluster. With HDFS, when the Dataproc cluster is deleted, the data stored in it also goes away unless I move it out. With GCS, the data lives independently of the cluster, so I can create ephemeral clusters on demand to process data and shut them down later without losing storage.

GCS also gives durability, high availability, and cheaper storage compared to managing HDFS disks. On top of that, it integrates natively with other GCP services like BigQuery and Dataflow, making it more versatile for pipelines. That's why Dataproc clusters are usually configured with GCS as the main storage layer instead of HDFS.

22. How does Dataproc read and write data directly from GCS?

Dataproc uses a connector called the GCS Hadoop connector (sometimes referred to as the gs:// connector). This allows Spark, Hive, and Hadoop jobs on Dataproc to treat GCS buckets like a native filesystem. So instead of hdfs:// paths, I just use gs:// paths in code or configuration.

For example, Spark can read with spark.read.parquet("gs://bucket/data/") or write with df.write.json("gs://bucket/output/"). Behind the scenes, the connector handles authentication, parallel I/O, and retries, so jobs can work with GCS just like HDFS.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

23. What are the trade-offs between HDFS and GCS in Spark jobs?

HDFS is faster for very small, frequent reads and writes because it is tightly coupled with the cluster's local disks and supports low-latency operations. GCS, being an object store, has slightly higher latency for metadata operations and doesn't support features like append.

But the trade-off is that GCS is much more scalable, durable, and cost-effective. With HDFS, I need to manage disk capacity, replication, and backups, while with GCS all of that is handled automatically.

So the trade-off is: use HDFS if ultra-low latency is required and the cluster is long-lived. Use GCS for almost all other cases because it gives persistence beyond the cluster, scales effortlessly, and is cheaper to maintain.

24. How does the small-file problem affect Dataproc jobs with GCS?

The small-file problem happens when there are thousands or millions of tiny files stored in GCS. Each file access in GCS has overhead, so Spark jobs end up spending more time listing and opening files than actually processing data. This causes high job latency, stragglers, and sometimes even job failures due to too many metadata operations.

For example, if log data is written as one file per event, Dataproc jobs will run very slowly when aggregating those logs. A better practice is to batch small files into larger ones, like 100MB–1GB per file, before storing them in GCS. This way, Spark can process data in parallel without wasting time on excessive file operations.

25. Why are Parquet or ORC preferred formats when storing processed data in GCS for Spark?

Parquet and ORC are columnar formats, which means they store data by columns instead of rows. This makes them very efficient for analytical queries where Spark jobs only need a few columns. Reading fewer columns reduces I/O and speeds up processing.

They also support compression and encoding techniques that reduce file size and improve scan performance. Another advantage is that they preserve schema and support nested structures, which makes data easier to integrate with BigQuery and other downstream tools.

Compared to CSV or JSON, which are row-based and uncompressed, Parquet and ORC use less storage, are faster to read, and are better optimized for parallel processing. That's why they are the standard choice for storing curated or processed data in GCS for Spark workloads.

26. How can you trigger a Composer DAG when a new file lands in GCS?

The common way is to use GCS-to-Pub/Sub notifications. I enable notifications on the bucket so that whenever a new file is uploaded, a Pub/Sub message is published. Then I configure my Composer environment to listen to that Pub/Sub topic using a TriggerDagRunOperator or a sensor. This way, the DAG is triggered automatically when new data arrives.

Another option is to use a GCSObjectExistenceSensor or GCSObjectUpdateSensor inside the DAG, but that involves polling, which is less efficient. The Pub/Sub-triggered approach is event-driven and preferred for near real-time pipelines.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

27. What operator would you use to move files across buckets after processing?

I would use the GCSToGCSOperator in Airflow (Composer). This operator allows me to copy or move objects between buckets or within the same bucket. For example, after ETL is complete, I can move the processed file from a staging bucket to an archive bucket.

If I only need to delete or clean up files, I can also use the GCSDeleteObjectsOperator. But for moving between buckets, GCSToGCSOperator is the right choice.

28. Why might you archive raw data into another bucket after ETL?

Archiving raw data provides a safety net. If something goes wrong in the ETL process or downstream consumers report issues, I can always go back to the original unmodified data. Keeping raw data separate also helps with compliance, auditing, and replaying pipelines in case of schema changes or new business logic.

Archiving into a different bucket also allows me to apply cheaper storage classes like Nearline or Coldline, so I save costs while still retaining the raw data for future use. This separation between raw and processed zones makes the pipeline more reliable and easier to maintain.

29. How can you enforce schema validation in Composer DAGs with GCS ingestion?

Inside the DAG, I can add a validation task before loading data. For example, I can use a PythonOperator to check if the incoming file matches the expected schema. The script can validate field names, data types, mandatory columns, and even row counts.

Another approach is to call a Dataflow job from Composer that validates schema during ingestion. If the schema does not match, the DAG can stop further tasks and alert the team.

By enforcing schema validation in the DAG, I make sure that only correct and compatible data gets loaded into the curated or processed zones, reducing the risk of failures later.

30. How would you orchestrate a pipeline from GCS → BigQuery using Composer?

I would design the DAG with the following steps:

1. A sensor task to detect when the new file lands in GCS (either through Pub/Sub trigger or a GCS sensor).
2. A validation step using a PythonOperator or Dataflow to check schema and data quality.
3. A load task using the GCSToBigQueryOperator to load the file directly from GCS into a BigQuery table. This operator supports CSV, JSON, Avro, and Parquet.
4. Optionally, a transformation step using BigQueryOperator to clean, join, or aggregate data after loading.
5. A final task to move the raw file into an archive bucket using GCSToGCSOperator.

This orchestration ensures the pipeline is automated, validated, and keeps both raw and processed data safe for future use.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

31. Why are Parquet and Avro preferred over CSV for large-scale data pipelines?

CSV is simple and human-readable, but it's inefficient for big data pipelines. It has no schema, so every time a system reads a CSV, it has to parse the text and infer data types, which slows things down. CSV also takes up more space because it's plain text without compression.

Parquet and Avro are preferred because they are binary formats, support schema, and compress well. Avro is great for row-based storage with schema evolution, while Parquet is columnar, which makes analytical queries much faster since only the needed columns are read. Both formats are more efficient for storage, faster to process, and integrate well with tools like Spark and BigQuery.

32. What are the trade-offs between row-based and columnar file formats in GCS pipelines?

Row-based formats like Avro are best for write-heavy and streaming scenarios. They store data row by row, so writing is fast, and reading full rows is efficient. They are good for transactional or real-time pipelines where I need complete records.

Columnar formats like Parquet and ORC are optimized for analytics. They store data by column, so queries that need only a few columns can skip the rest. This reduces I/O and makes scans much faster. The trade-off is that writing is slower because data has to be organized by column.

So the decision is based on workload: row-based for ingestion and streaming, columnar for querying and analytics.

33. How does schema evolution work in Avro or Parquet?

Schema evolution means handling changes in the data schema over time without breaking pipelines. Avro and Parquet both support it, but in slightly different ways.

In Avro, the schema is stored with the data, so readers can use a different schema as long as it is compatible. For example, I can add new optional fields, remove unused ones, or rename with aliases. It's very flexible and commonly used for evolving pipelines.

In Parquet, the schema is part of the metadata. Adding new columns is supported, and old files can still be read with missing columns treated as null. But changing data types is harder.

In practice, both formats make it possible to add new fields while keeping old data valid, which is important for long-running data lakes.

34. Why is Snappy compression often used in GCS-based pipelines?

Snappy is popular because it provides a good balance between compression ratio and speed. It doesn't compress as much as GZIP, but it is much faster to compress and decompress. That means less CPU overhead during reads and writes, which is important for high-throughput pipelines.

When storing large datasets in GCS for Spark or BigQuery, using Snappy with Avro or Parquet reduces storage costs while still keeping query performance fast. BigQuery also supports Snappy natively, which makes it a safe choice for interoperability across systems.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

35. How do you decide the right file format based on the downstream system (BigQuery, Spark, ML)?

The choice depends on how the data will be consumed:

- For BigQuery, Parquet and Avro are best because BigQuery can load and query them natively and efficiently. They save cost since only needed columns are scanned.
- For Spark, both Avro and Parquet are good. Parquet is ideal for analytics, while Avro works better if the data is being written quickly in streaming jobs.
- For ML workloads, Parquet is useful if the pipeline is feature-heavy and only certain columns are needed. Avro can also work well if the pipeline is handling row-based event data.
- If data needs to be shared with external systems or manually inspected, CSV or JSON may still be used, but usually only at the raw layer.

So the decision is guided by the nature of queries, the downstream engine, and whether schema evolution is required.

36. How can you partition GCS data by date or region for better query performance?

Partitioning in GCS is done by designing the folder structure in a way that reflects common query filters. For example, if queries usually filter by date, I store files in paths like gs://bucket/sales/year=2025/month=09/day=11/. Similarly, if queries filter by region, I can organize data like gs://bucket/sales/region=US/ or region=EU/.

This structure makes it easier for BigQuery, Spark, or Hive jobs to read only the relevant partitions instead of scanning the entire dataset. Partitioning also helps manage lifecycle policies, because I can set different retention or storage classes at the folder level.

37. What are hive-style partitions, and why are they useful in GCS?

Hive-style partitions are a folder naming convention where each partition is written in the format column=value. For example: gs://bucket/events/date=2025-09-11/region=us/.

They are useful because many engines like Hive, Spark, and BigQuery can automatically detect partitions from the folder names without needing extra metadata tables. This makes queries faster because the engine can prune partitions just by looking at the path.

It's a simple but powerful way to make data lakes more structured and query-friendly.

38. How does partition-aware ingestion help when loading GCS data into BigQuery?

Partition-aware ingestion means mapping the folder structure in GCS directly to partitions in BigQuery. For example, if files are stored under date=2025-09-11/, BigQuery can automatically load those into the right partition of a partitioned table.

This helps avoid full-table scans because queries can target specific partitions. It also makes ingestion pipelines more efficient, since I don't need to manually add partition filters.

With partition-aware ingestion, data freshness and query cost are both improved, as BigQuery only processes the partitions relevant to the query.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

39. Why is directory structure important when designing data lakes in GCS?

Even though GCS is an object store and not a true filesystem, the way I design the directory structure acts like metadata for the data. A clear directory structure makes it easy to discover, filter, and manage data.

For example, if the structure is zone/dataset/year/month/day/, I can quickly identify where raw, curated, or processed data is kept, and I can run lifecycle policies by directory. Downstream tools like Spark, Hive, and BigQuery also use the folder hierarchy to optimize queries.

Without a good structure, the data lake becomes a "data swamp" where it's hard to find or manage data, and query performance suffers.

40. What are the pros and cons of fine-grained vs coarse-grained partitioning in GCS?

Fine-grained partitioning means breaking data into very small partitions, like by hour or by customer ID. The advantage is that queries can be extremely selective and efficient if they only need a small subset. The downside is that it can create too many small files and metadata overhead, which slows down jobs.

Coarse-grained partitioning means larger partitions, like by month or by region. The advantage is fewer files and less overhead, but queries may end up scanning more data than necessary because partitions are broader.

In practice, I balance the two. For high-volume data like logs, daily partitions work well. For lower-volume data, monthly partitions may be enough. The goal is to choose a partitioning strategy that balances query efficiency and file management.

41. Why should a GCS-based data lake have raw, curated, and analytics zones?

Separating data into zones makes the lake organized and easier to govern. The raw zone stores the original source data exactly as received. This is important because it acts as the single source of truth, and I can always reprocess it if something goes wrong downstream.

The curated zone is where data is cleaned, validated, and standardized. Here I remove duplicates, fix schema issues, and transform data into a consistent format. This ensures that downstream teams don't have to work with messy data.

The analytics zone (sometimes called processed or consumption zone) is where I keep aggregated or transformed data that is ready for direct use in reports, dashboards, or ML models.

This layered approach separates responsibilities, applies different access controls, and makes the pipeline more robust and auditable.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

42. What is schema-on-read, and how does it apply to GCS data lakes?

Schema-on-read means the data is stored in its raw form without enforcing a schema at the time of ingestion. The schema is applied later, when a system like Spark or BigQuery reads the data.

In GCS data lakes, this is common because raw files can be CSV, JSON, or logs with flexible structures. By not enforcing schema upfront, I can ingest data quickly without worrying about immediate compatibility. Then, when analysts or pipelines need the data, they define the schema on read.

The benefit is flexibility, but the trade-off is that bad data might sneak in and cause issues later if schema validation is not applied carefully in the curated zone.

43. How do you manage slowly evolving schemas in a GCS-based data lake?

Slowly evolving schemas are handled using formats that support schema evolution, like Avro or Parquet. These allow me to add new optional fields without breaking old data.

At the pipeline level, I maintain schema versions and keep raw data unchanged so that I can always replay with a new schema if needed. In the curated zone, I enforce schema compatibility, usually allowing field additions but not destructive changes.

For downstream systems like BigQuery, I enable schema update options such as allowing new nullable columns during load. I also document schema changes and notify downstream teams so they can adapt queries.

The key is to design for backward compatibility and avoid breaking pipelines when fields are added or changed slowly over time.

44. What is the difference between incremental snapshots vs full reloads in GCS pipelines?

An incremental snapshot means only the new or changed data since the last run is ingested and stored. For example, loading just yesterday's sales records instead of reloading the entire sales history. This saves time and cost because less data is moved and processed.

A full reload means bringing the entire dataset every time. This ensures the target is always consistent but is expensive and slow for large datasets.

In practice, I prefer incremental snapshots for efficiency, and I use full reloads only in special cases like small reference tables, initial loads, or when the source system doesn't support incremental extraction.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

45. Why is GCS often used as a staging layer before BigQuery ingestion?

GCS is a cheap, durable, and scalable place to land files before loading them into BigQuery. Using GCS as staging decouples ingestion from transformation. Source systems can dump raw files into GCS at any time, and then scheduled pipelines can pick them up for validation and loading into BigQuery.

This approach allows schema checks, data quality validation, and transformations to happen before data reaches BigQuery. It also provides an audit trail since raw files remain in GCS even after loading.

Another reason is cost. If something goes wrong, I can reprocess files from GCS without re-extracting from the source system, saving both effort and compute cost.

46. How do GCS notifications integrate with Pub/Sub?

GCS can be configured to publish an event message to a Pub/Sub topic whenever a change happens in a bucket, like when a new file is uploaded or an object is deleted. This message contains metadata about the event, such as the file name, bucket name, and time.

Other services like Cloud Functions, Cloud Run, or Dataflow can then subscribe to that Pub/Sub topic and take action automatically. This integration allows event-driven architectures where GCS acts as the source of events and Pub/Sub distributes them to downstream consumers.

47. How can you trigger Dataflow or Composer pipelines when a new file lands in GCS?

For Dataflow, I can subscribe a pipeline directly to the Pub/Sub topic that receives GCS events. Whenever a file lands, Pub/Sub sends a message, and Dataflow picks it up to start processing.

For Composer, I use a DAG trigger that listens to Pub/Sub. When a file upload event happens, the Pub/Sub message can start the Composer DAG automatically. Alternatively, I can use a GCS sensor inside a DAG, but Pub/Sub-based triggering is more efficient and event-driven.

This setup ensures pipelines start automatically as soon as new data arrives, reducing manual scheduling.

48. How can event-driven ingestion automate BigQuery loads?

The pattern is usually: GCS triggers a Pub/Sub message when a new file is uploaded, then a Cloud Function or Cloud Run service listens to that event and kicks off a BigQuery load job. The function can validate the file format, schema, and then call the BigQuery API to load the file into the correct table or partition.

This removes the need for batch scheduling and makes data available in BigQuery almost immediately after it lands in GCS. It's especially useful for streaming-style use cases where freshness is important.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

49. What is a typical use case of real-time ingestion pipelines using GCS events?

A common use case is log or sensor data pipelines. For example, IoT devices or applications generate logs continuously and write them into GCS in near real-time. Each file upload triggers a Pub/Sub event, which in turn starts a pipeline to validate, transform, and load the data into BigQuery or another storage system.

Another example is financial transactions or clickstream data. As soon as raw files land in GCS, event-driven triggers ensure the data is processed and available quickly for fraud detection, monitoring, or analytics.

50. How do you handle errors when event triggers fail in GCS pipelines?

Error handling is very important in event-driven pipelines. I usually set up a dead-letter topic (DLQ) in Pub/Sub so that if a message cannot be processed, it is sent to DLQ for later inspection. This prevents data loss.

In addition, I log all errors to Cloud Logging and set up alerts in Cloud Monitoring so the team is notified when failures happen. For transient errors like network issues, retries with exponential backoff are usually enough.

For permanent errors, I keep the raw files in GCS so I can reprocess them manually or through a recovery pipeline. This way, no data is lost even if the trigger fails.

51. When is GCS best used as a batch staging area?

GCS is best as a batch staging area when data arrives in bulk or at scheduled intervals rather than continuously. For example, daily sales reports, nightly database dumps, or end-of-day transaction files from on-prem systems. These files can land in GCS first and then be picked up by ETL pipelines into BigQuery or Dataproc.

It's also a good choice when data needs to be validated, archived, or transformed before further processing. Using GCS as staging decouples ingestion from transformation, so even if the downstream job fails, the raw data is safe in GCS and can be retried without going back to the source.

52. How can GCS act as a sink for streaming pipelines?

In streaming pipelines, GCS is often used to store data in time-based windows. For example, a Dataflow pipeline consuming from Pub/Sub can write files into GCS every 5 minutes or every hour, using windowing to group the data.

This way, GCS holds micro-batches of streaming data, which can later be processed by batch-oriented systems like Spark, Dataproc, or BigQuery ingestion jobs. GCS acts as a durable, scalable sink that converts continuous streams into manageable chunks of files.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

53. What are the trade-offs between batch vs streaming pipelines with GCS?

With batch pipelines, the advantage is simplicity and efficiency. Data arrives at scheduled times, is processed in bulk, and written to GCS in large files. This reduces overhead and storage costs. The drawback is that data is not immediately available; there's usually a delay until the next batch runs.

With streaming pipelines, the advantage is low latency and near real-time availability of data. GCS gets updated files quickly, which supports use cases like monitoring or fraud detection. The drawback is higher complexity, more small files, and higher compute cost to keep pipelines running continuously.

So the trade-off is speed versus simplicity and cost.

54. How can you combine Dataflow streaming with GCS batch processing?

A common pattern is to use Dataflow streaming to collect raw events and write them into GCS in small time-based files. Then, batch jobs (using Dataproc, Dataflow batch, or Composer DAGs) pick up those files later for heavy transformations or loading into BigQuery.

For example, IoT sensor data streams into Pub/Sub, Dataflow streaming writes 5-minute Avro files into GCS, and then every hour a batch pipeline aggregates those files into curated datasets in BigQuery.

This hybrid approach combines the low-latency benefits of streaming with the efficiency and reliability of batch analytics.

55. How does data freshness requirement affect the choice of batch vs streaming in GCS?

If the business needs near real-time insights, like fraud detection, website personalization, or operational monitoring, streaming pipelines with GCS as a sink are preferred. This ensures the data is available within minutes or seconds.

If the use case is more analytical and doesn't need immediate data, like monthly reports, daily dashboards, or trend analysis, then batch pipelines are sufficient. They are cheaper and simpler to manage.

So the data freshness requirement is the main deciding factor: tighter freshness means streaming, while relaxed freshness allows batch. Many companies even mix both, using streaming for real-time needs and batch for historical aggregation.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

56. How can you validate schema before ingesting GCS files into BigQuery?

Before loading into BigQuery, I usually add a validation step in the pipeline. One way is to use a Python or Dataflow job to check that the file columns, data types, and delimiters match the expected schema. For example, I can compare the file's header against the target BigQuery table schema.

BigQuery also supports schema auto-detection, but in production, I prefer explicitly defining schemas to avoid surprises. If the schema doesn't match, the file can be quarantined into a "rejects" folder in GCS and flagged for review, while valid files continue through the pipeline.

This ensures that only compatible data gets ingested into BigQuery.

57. How can you detect corrupt or incomplete files in GCS?

Corrupt or incomplete files can be detected by checking file size, row count, or checksum before ingestion. For example, if I expect daily files of around 500MB, and one file is only a few KB, that's a sign it might be incomplete.

Some pipelines generate checksum or hash files alongside the data. I can validate these to ensure the file is complete and hasn't been altered. Another option is to check for consistent row counts or mandatory columns before processing.

If a file fails these checks, it can be flagged, quarantined, or retried.

58. What strategies can you use to handle duplicate file uploads in GCS?

One common strategy is to design idempotent pipelines, where reprocessing the same file doesn't create duplicate rows in the target. For example, using primary keys or deduplication logic in BigQuery after loading.

Another approach is to enforce unique file naming conventions that include identifiers like date and batch number. If a file with the same name already exists, the new one can be rejected or placed in a duplicates folder.

Metadata tracking also helps — by recording which files have already been processed in a metadata table, the pipeline can skip duplicates safely.

59. How can Dataflow or Composer help automate data quality checks for GCS pipelines?

Dataflow can run validation jobs before loading data. For example, it can check schema consistency, apply regex checks on fields, validate null values, and log errors to a separate GCS path. This makes quality checks scalable for large datasets.

Composer, on the other hand, can orchestrate a DAG that includes data validation tasks. For example, a PythonOperator in the DAG can check file headers, and if validation fails, the DAG can stop further tasks and notify the team.

Together, Dataflow ensures checks at scale, while Composer ensures checks are part of the overall workflow.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

60. Why is it useful to store validation logs alongside data in GCS?

Keeping validation logs next to the data makes troubleshooting much easier. If a file is rejected, I can quickly check its log in the same location to see why. This creates a transparent audit trail, which is important for compliance and debugging.

It also helps downstream teams. If someone consumes data from the curated or analytics zone, they can look at the validation logs to understand what was excluded and why.

Storing logs with data also ensures that if the pipeline is replayed in the future, the validation history is available, keeping the data lake consistent and trustworthy.

61. Why are too many small files a problem in Spark, Dataflow, or BigQuery pipelines?

Small files create overhead because every file has to be opened, read, and processed individually. In Spark and Dataflow, this increases job latency since workers spend more time on metadata operations rather than actual data processing. It can also cause stragglers when some tasks handle thousands of small files while others process fewer.

In BigQuery, too many small files loaded into a table create inefficient partitions and can drive up query costs, since BigQuery still has to scan metadata for all files. In short, small files waste resources, slow down pipelines, and increase costs.

62. What are common strategies to merge or compact small files in GCS?

A common approach is to run compaction jobs that periodically merge small files into larger ones. For example, a Dataflow or Spark job can read many small files and write them back as fewer large files (100MB–1GB each).

Another strategy is to use tools like BigQuery's external tables temporarily, then export them back to GCS in optimized Parquet or Avro files. Lifecycle policies can also be set to move small raw files into an archive bucket while the compacted files are stored in the curated zone.

The goal is always to balance file size so they are big enough for efficiency but not so large that they reduce parallelism.

63. How can Dataflow sinks be tuned to reduce the number of small files in GCS?

In Dataflow, I can control the number of output shards when writing to GCS. By default, Dataflow may create many shards, which means many small files. By explicitly setting the number of shards, I can reduce the number of files and make them larger.

Windowing also plays a big role in streaming pipelines. If the window size is too small, files are written too frequently, creating small files. Increasing the window size results in fewer, larger files.

Compression and choosing efficient formats like Parquet or Avro also help reduce file count while optimizing storage.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

64. How does partition tuning help reduce small file impact in BigQuery loads?

If GCS files are loaded into BigQuery partitions that are too fine-grained, like hourly partitions for low-volume data, each partition ends up with very few rows and many small files. This increases storage overhead and makes queries inefficient.

By tuning partitions to a more suitable level, like daily instead of hourly, the data per partition becomes larger and more efficient. This reduces the number of load jobs and avoids scattering data into too many small partitions.

So careful partition design aligns data volume with partition size, reducing the small-file problem downstream in BigQuery.

65. How can you design ingestion pipelines to avoid generating small files in GCS?

The key is to batch data before writing to GCS. For streaming data, I use windowing to group events into time buckets (for example, 15 minutes) so that files are larger. For batch ingestion, I combine files on-prem before uploading instead of sending thousands of tiny files.

I also choose efficient file formats like Parquet or Avro, which naturally hold more data in a single file compared to CSV. Another strategy is to design upstream systems to generate fewer, bigger files rather than many tiny ones.

Finally, I enforce compaction as part of the pipeline design — raw files may come in small, but before promoting them to curated, I merge them into optimal sizes. This ensures downstream tools like Spark and BigQuery work efficiently.

66. How do lifecycle policies in GCS help manage raw and processed data?

Lifecycle policies in GCS let me automatically move or delete objects based on rules like age, storage class, or version. For raw data, I usually keep it for a fixed period (say 90 days) and then move it to cheaper storage or delete it if not needed anymore. For processed data, I may keep it in standard storage for quick access but move older data to Nearline or Coldline.

This automation prevents the bucket from filling up with unnecessary files, reduces manual work, and ensures compliance with retention requirements. It also makes the data lake cleaner and more cost-efficient.

67. Why is archiving processed data into Coldline/Archive storage useful?

Processed data may not be queried frequently after some time, but it still needs to be retained for regulatory or historical purposes. Moving it into Coldline or Archive storage lowers storage costs significantly while still keeping the data accessible if needed.

For example, monthly financial reports or historical IoT data may not be accessed often, but they must be kept for years. Archiving them in cheaper classes ensures compliance while optimizing cost. If needed, they can still be restored, though with some retrieval latency.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

68. What retention policies should you apply to staging data in GCS?

Staging data is usually temporary, so I apply short retention policies, often 7–30 days, depending on the pipeline. This ensures that temporary or intermediate files don't accumulate and increase storage costs.

I configure lifecycle rules to automatically delete staging files after the retention period. Since staging data can always be regenerated from raw sources, there is no need to keep it long-term. This keeps the staging zone clean and efficient.

69. How can you automate cleanup of temporary data in GCS pipelines?

The simplest way is to use GCS lifecycle rules that automatically delete objects after a set number of days. For example, I can configure a rule that deletes files in a temporary folder after 3 days.

Another option is to add cleanup tasks inside orchestration tools like Composer, where a DAG step deletes temporary files once processing is complete. This gives more control when specific cleanup timing is needed.

Automation ensures that temp files don't accumulate and pipelines remain cost-effective without requiring manual intervention.

70. How do lifecycle rules help optimize cost in GCS-based ETL pipelines?

Lifecycle rules move data to cheaper storage classes or delete it when it's no longer needed, which directly reduces storage costs. For example, raw data can automatically move from Standard to Nearline after 30 days, and then to Coldline after 90 days.

They also reduce the cost of managing large buckets by preventing old or unused files from piling up. This means I don't pay for storing data that has no business value.

In short, lifecycle rules enforce cost discipline by automatically applying data retention and tiering strategies, which is especially important in large-scale ETL pipelines where data grows very quickly.

71. How can GCS act as a staging layer for BigQuery data warehouses?

GCS is often the landing zone for data before it enters BigQuery. Source systems can dump files (CSV, JSON, Parquet, Avro) into GCS. From there, I can apply schema validation, quality checks, and transformations before loading into BigQuery.

This staging layer is useful because it decouples ingestion from BigQuery loads. If something goes wrong, I can always reprocess the files from GCS instead of re-extracting from the source. It also provides a historical audit trail of raw data, which is important for debugging and compliance.

In short, GCS staging ensures reliability, cost control, and flexibility before data is promoted to the warehouse.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

72. What is a typical IoT ingestion pipeline using Pub/Sub, Dataflow, and GCS?

A common IoT pipeline works like this: devices send events or telemetry data continuously. These events are published into Pub/Sub topics. Dataflow then consumes these events in real time, applies transformations or enrichments, and writes them into GCS in time-windowed files (like Avro or Parquet every 5 minutes).

Later, batch jobs or scheduled pipelines load these files from GCS into BigQuery for analytics or into ML pipelines for model training.

This pattern ensures scalability (Pub/Sub), processing flexibility (Dataflow), and durable, cheap storage (GCS).

73. How can ML pipelines use GCS for feature storage?

In ML workflows, GCS often stores curated features in formats like Parquet or TFRecords. Raw data is ingested and transformed into structured feature sets, which are then written to GCS for training and inference pipelines.

During training, ML frameworks like TensorFlow or PyTorch can read directly from GCS. For serving, feature stores can either reference data in GCS or load it from GCS into a more optimized system.

GCS is useful here because it's scalable, integrates with Vertex AI and Dataproc, and supports versioning of feature sets. This makes experiments reproducible and models more reliable.

74. How would you migrate an on-prem Hadoop data lake into GCS?

I would approach this in phases. First, I'd analyze the existing Hadoop data lake — file formats, volume, partitioning, and how it's being consumed. Then, for migration, I can use tools like DistCp (distributed copy) to move HDFS data into GCS via the GCS connector. For very large datasets, Google's Transfer Appliance can also be used.

Next, I'd restructure the data in GCS into raw and curated zones, possibly converting row-based formats (CSV/JSON) into Parquet or Avro for efficiency. I'd also reconfigure Hadoop/Spark jobs to point to gs:// paths instead of hdfs://.

Finally, I'd validate the migrated data, set up IAM for access control, and establish lifecycle policies. This phased approach ensures minimal downtime and a smooth transition.

75. How would you design a multi-team GCS data lake with raw and curated zones?

I would start by creating a central bucket (or multiple buckets) with a clear zone structure: raw, curated, and analytics. Each team would have its own subdirectories within these zones. For example: gs://datalake/raw/teamA/, gs://datalake/curated/finance/.

Access control would be applied at the folder or prefix level using IAM policies, so teams can write to their raw zones but only read curated data from other teams. Sensitive data could be further protected with VPC-SC or CMEK encryption.

I'd also enforce naming standards, lifecycle policies, and schema validation rules to keep the lake consistent. Composer or Dataflow could orchestrate pipelines that promote data from raw to curated, ensuring data quality before cross-team sharing.

This design allows collaboration while keeping data secure, organized, and reliable.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar